

CH

MapReduce & YARN

- ☐ MRv1 : rappel, architecture, limitations
- ☐ YARN vs MRv1
- ☐ YARN : Architecture et fonctionnement
- ☐ YARN : Allocation des ressources
- ☐ Planification des ressources Hadoop



Mastère DS, 1 année

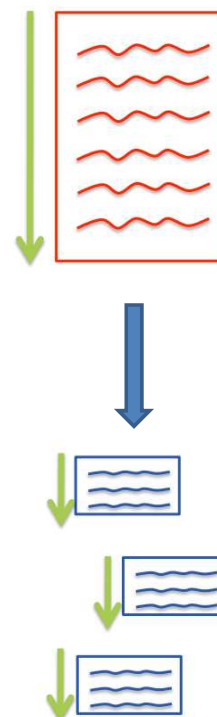
FSB Bizerte, Anis Ben Aicha

1

MRv1-Rappel : idée et principe



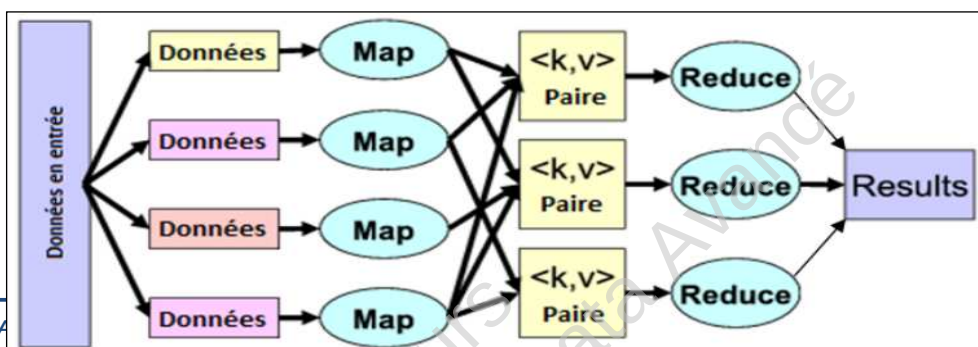
- Algorithme conventionnel :
 - **Traitement séquentiel**
 - ✓ Très lent
 - ✓ Ressources verticales importantes
 - ✓ Pas adéquat pour applications temps réel
- Paradigme Mapreduce :
 - **Traitement parallèle**
 - ✓ Très rapide
 - ✓ Cluster des machines de moyenne performances
 - ✓ Adéquat pour applications temps réel



MRv1-Rappel : idée et principe



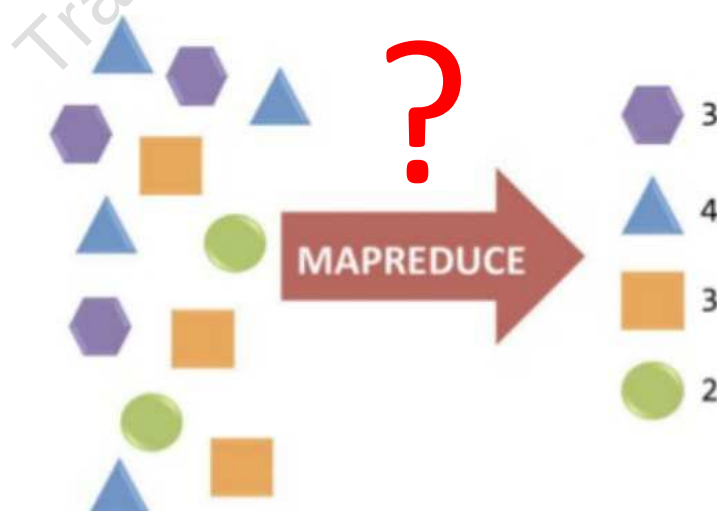
- Principe
 - Exécution d'un problème de manière distribuée
 - Découpage en sous-problèmes
 - Execution des sous-problèmes sur les différentes machines du cluster
 - Stratégie algorithmique dite du Divide and Conquer
- Deux étapes principales :
 - Map: Emission de paires **<clé,valeur>** pour chaque donnée d'entrée lue
 - Reduce: Regroupement des valeurs de clé identique et application d'un traitement sur ces valeurs de clé commune



MRv1-Rappel : idée et principe



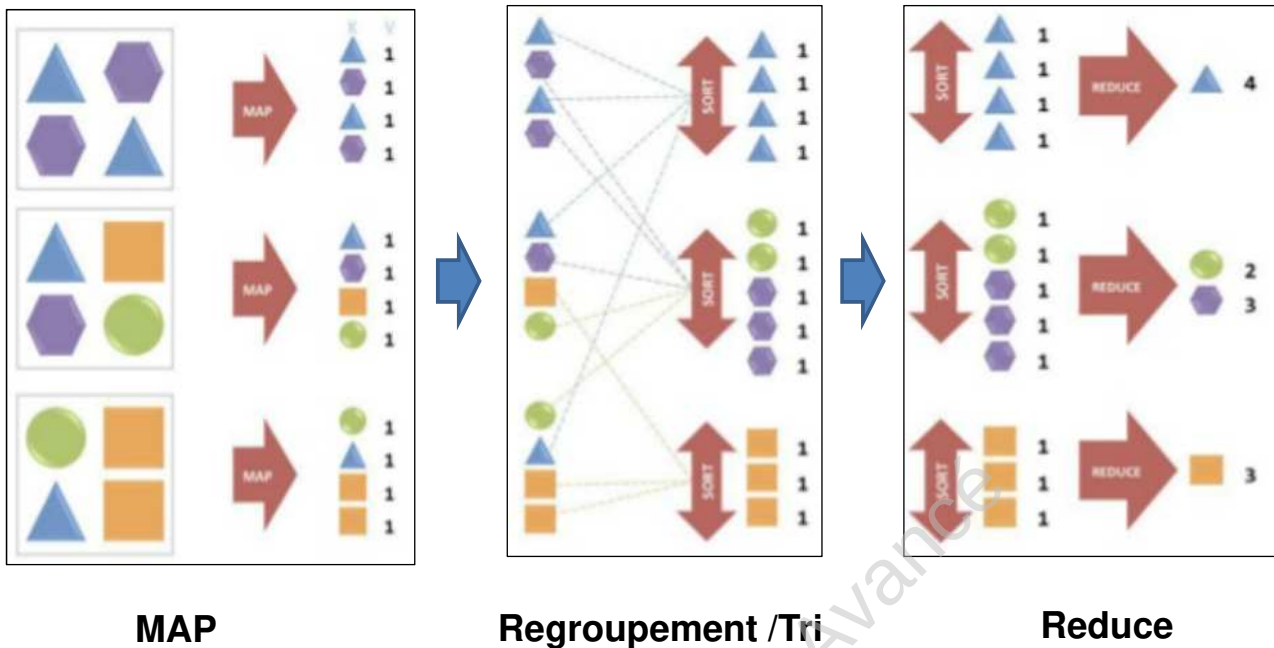
Exemple : visuel



MRv1-Rappel : idée et principe



Exemple : visuel



FSB Bizerte, Anis Ben Aïcha

5

Mastère DS, 1 année

MRv1-Rappel : idée et principe



- Ecrire un programme Map Reduce revient à :
 - 1 Choisir une manière de découper les données afin que les opérations Map et Reduce soient parallélisables
 - 2 Choisir la clé à utiliser pour le problème
 - 3 Écrire le programme pour l'opération Map
 - 4 Écrire le programme pour l'opération Reduce

FSB Bizerte, Anis Ben Aïcha

6

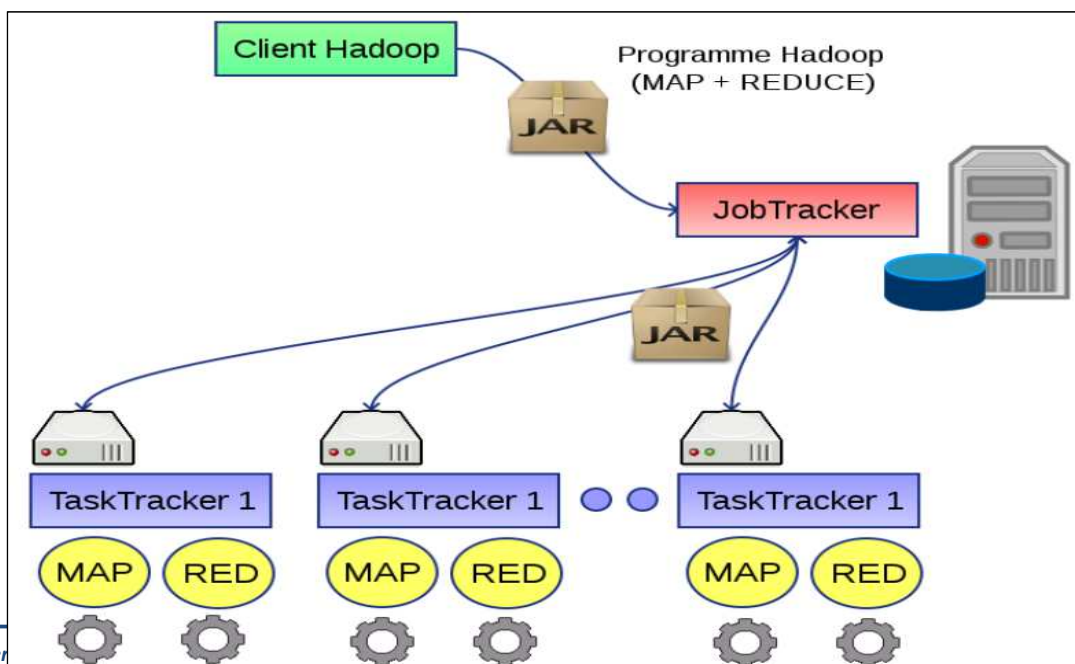
Mastère DS, 1 année

- Architecture : Repose sur deux serveurs:
 - Le JobTracker (unique sur le cluster).
Reçoit les tâches map/reduce à exécuter (sous la forme d'une archive Java.jar), organise leur exécution sur le cluster.
 - Le TaskTracker (plusieurs par cluster).
Exécute le travail map/reduce (sous la forme de tâches map et reduce ponctuelles avec les données d'entrée associées).
Chacun des TaskTrackers constitue une unité de calcul du cluster.

MRv1-Rappel : Architecture



- Le serveur JobTracker est en communication avec HDFS; il sait où sont les données d'entrée du programme map/reduce et où doivent être stockées les données de sortie. Il peut ainsi optimiser la distribution des tâches selon les données associées.



MRv1-Rappel : Architecture



- JobTracker suit l'état des TaskTrackers par le biais de paquets *heartbeat* envoyés continuellement par les TaskTrackers.
- En cas de défaillance d'un TaskTracker (heartbeat manquant ou tâche échouée), le JobTracker redistribue la tâche à un autre nœud.
- Au cours de l'exécution d'une tâche, il est possible d'obtenir des statistiques détaillées sur son évolution (étape actuelle, avancement, temps estimé avant completion, etc.) et ceci par le biais du client console hadoop.
- Chaque nœud a un certain nombre de slots prédéfinis (Map Slots et Reduce Slots).
Un slot est une unité d'exécution qui représente la capacité du task tracker à exécuter une tâche (map ou reduce) individuellement, à un moment donné
- Remarque : Les deux serveurs « uniques » NameNode et JobTracker sont souvent actifs au sein d'une seule et même machine: le **nœud maître** du cluster.

MRv1-Rappel : Architecture



Les composants de MapReduce v1

- ✓ API : permet au programmeur d'écriture une applications MapReduce
- ✓ Framework : permettant l'exécution des Jobs Map, Shuffle/Sort et reduce
- ✓ Resource Management Infrastructure : gère les nœuds du cluster, alloue des ressources et assure l'ordonnancement des jobs



MRv1-Rappel : Limitations de MRv1

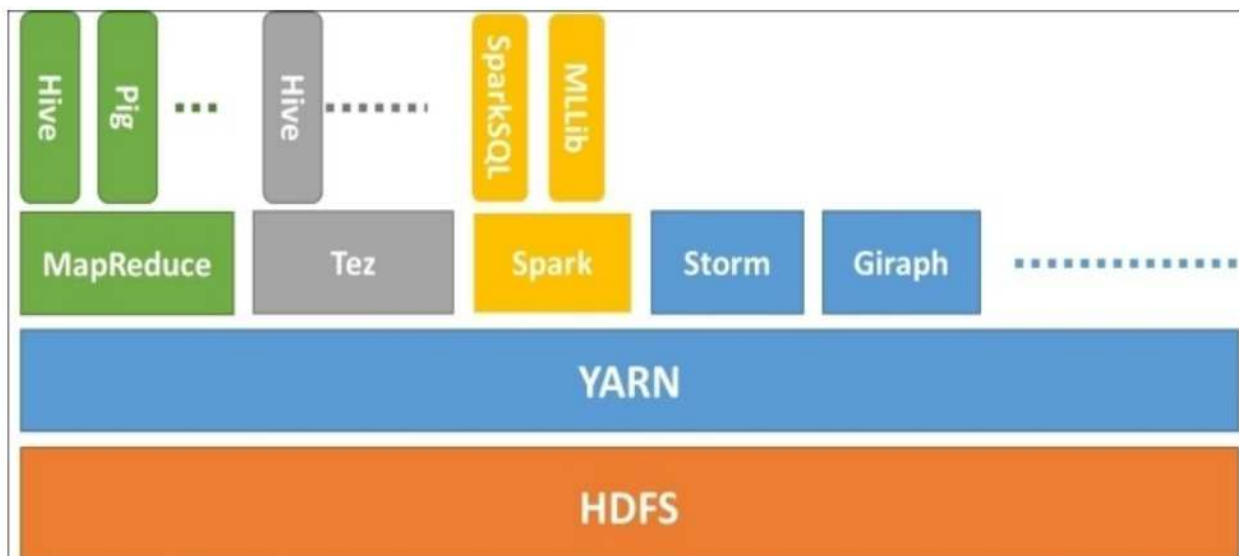


- Le serveur Job Tracker s'exécute sur la machine maitresse. En plus il assure plusieurs tâches (ordonnancement, gestion de ressources...)
 - ✓ Problème de scalabilité : Les performances de Job Tracker sont limitent le nombre de DataNodes supervisées (nombre de nœuds par cluster limité à 4000)
 - ✓ Job Tracker tombe en panne → réinitialisation de tous les Task Tracker (Problème de disponibilité)
 - ✓ Le nombre des **slots** est prédéfini → Problème d'exploitation des capacités des nœuds
 - ✓ Le Job Tracker est fortement intégré à Map Reduce → Problème d'interopérabilité (impossible d'exécuter des applications non-MapReduce sur HDFS)

YARN : objectif principal



- But de YARN
- Permet aux utilisateurs d'utiliser plusieurs applications distribuées qui cohabitent côte à côte avec des applications MapReduce et partageant le même cluster et le système de fichiers HDFS.



- Dans Mapreduce 1, il existe deux types de démon qui contrôlent le processus d'exécution des tâches : jobtracker et un ou plusieurs tasktrackers.
 - **Jobtracker** coordonne toutes les tâches exécutées sur le système en ordonnant les tâches à exécuter sur les Tasktrackers.
 - Les **Tasktrackers** exécutent les tâches et envoient des rapports d'avancement au jobtracker, qui conserve un enregistrement de l'avancement global de chaque tâche. Si une tâche échoue, le jobtracker peut la reprogrammer sur un autre tasktracker.
- Dans Mapreduce 1, le Jobtracker s'occupe de
 - la planification des tâches (appariement des tâches avec Suivi des tâches) et
 - le suivi de l'avancement des tâches (suivi des tâches, redémarrage défaillant ou lent).

- Contrairement au MRv1 les tâches assurées par Jobtracker sont séparées en deux :
 - le gestionnaire de ressources
 - application master (un pour chaque tâche Mapreduce).
- Le Jobtracker est également responsable de stockage de l'historique des travaux pour les travaux terminés,
- Remarque : il soit possible d'exécuter un serveur d'historique des travaux comme un démon distinct pour décharger le Jobtracker. Dans YARN, le rôle équivalent est assuré par un **serveur de timeline** (qui stocke l'historique des applications)

Comparaison des composants de MRv1 & YARN

MapReduce 1	YARN
Jobtracker	Resource manager, application master, timeline server
Tasktracker	Node manager
Slot	Container

▪ Scalabilité

- Dans MRv1, un seul Jobtracker trouve rapidement ces limites de gestion des tasktrakers (4 000 nœuds et 40 000 tâches). Ceci découle du fait que le Jobtracker doit gérer à la fois les tâches et ressources.
- YARN surmonte ces limites en Grâce à son architecture de gestion des ressources et de gestion des applications fractionnées (jusqu'à 10 000 nœuds et 100 000 tâches).
- Contrairement au Jobtracker de MRv1, chaque instance d'une application (ici, une tâche Mapreduce) a un maître d'application dédié, qui s'exécute pendant toute la durée de l'application.

▪ Disponibilité

- La haute disponibilité est habituellement obtenue en reproduisant (dupliquant) l'état démon afin de répondre au service demandé ou en cas d'échéance.
- Cependant, la grande quantité d'état complexe en évolution rapide dans le mémoire du Jobtracker (chaque état de tâche est mis à jour toutes les quelques secondes) rend très difficile la mise à niveau de HA dans le service Jobtracker.
- Avec les responsabilités du Jobtracker réparties entre le gestionnaire de ressources et application master, la haute disponibilité est obtenue

MRv1 Vs YARN : Avantages



■ Utilisation

- Dans Mapreduce 1, chaque Tasktracker est configuré avec une allocation statique de taille fixe « slot », qui sont divisés en **Map slots** et **Reduce slots** au moment de la configuration.
- Un Map (res. Reduce) slot ne peut être utiliser que pour exécuter une tâche Map (res. Reduce)
- Dans YARN, **Resource manager** gère un pool de ressources plutôt qu'un nombre fixe de slots.
- Mapreduce fonctionnant sur YARN n'aura pas la contrainte où une tâche Reduce doit attendre parce que seuls des slots Map sont en cours d'exécution (cas fréquent dans MRv1)
- Avec MRv2, si les ressources d'un nœud pour exécuter une tâche sont disponibles, alors la sera exécutée.
- Les ressources de YARN ne sont pas affectées d'une façon statique (slot). Une application peut faire des demande de ressources selon ses besoins.
- L'allocation statique des ressources dans MRv1 (slot indivisibles) : si trop grande → gaspillage de ressources ou trop faible → peut causer un échec pour la tâche en question.

MRv1 Vs YARN : Avantages



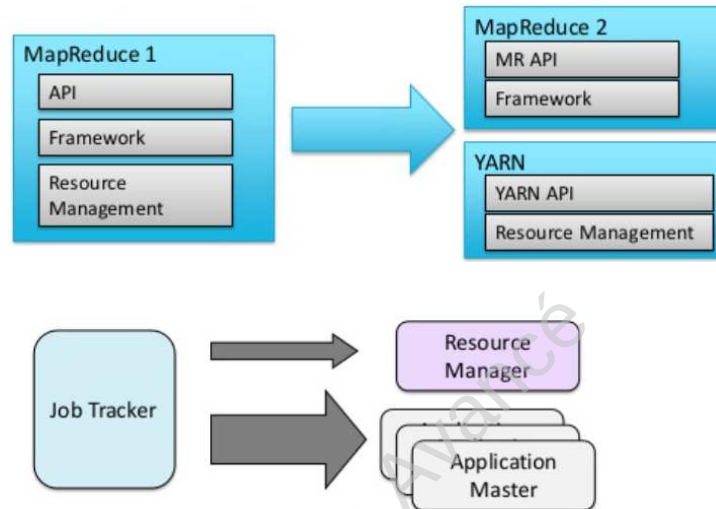
■ Multilocal

- Le plus grand avantage de YARN est qu'il ouvre Hadoop à d'autres types de application distribuée au-delà de Mapreduce.
- Mapreduce n'est qu'une application YARN parmi beaucoup.
- Il est même possible pour les utilisateurs d'exécuter différentes versions de Mapreduce sur la même cluster YARN, ce qui rend le processus de mise à niveau de Mapreduce plus gérable.

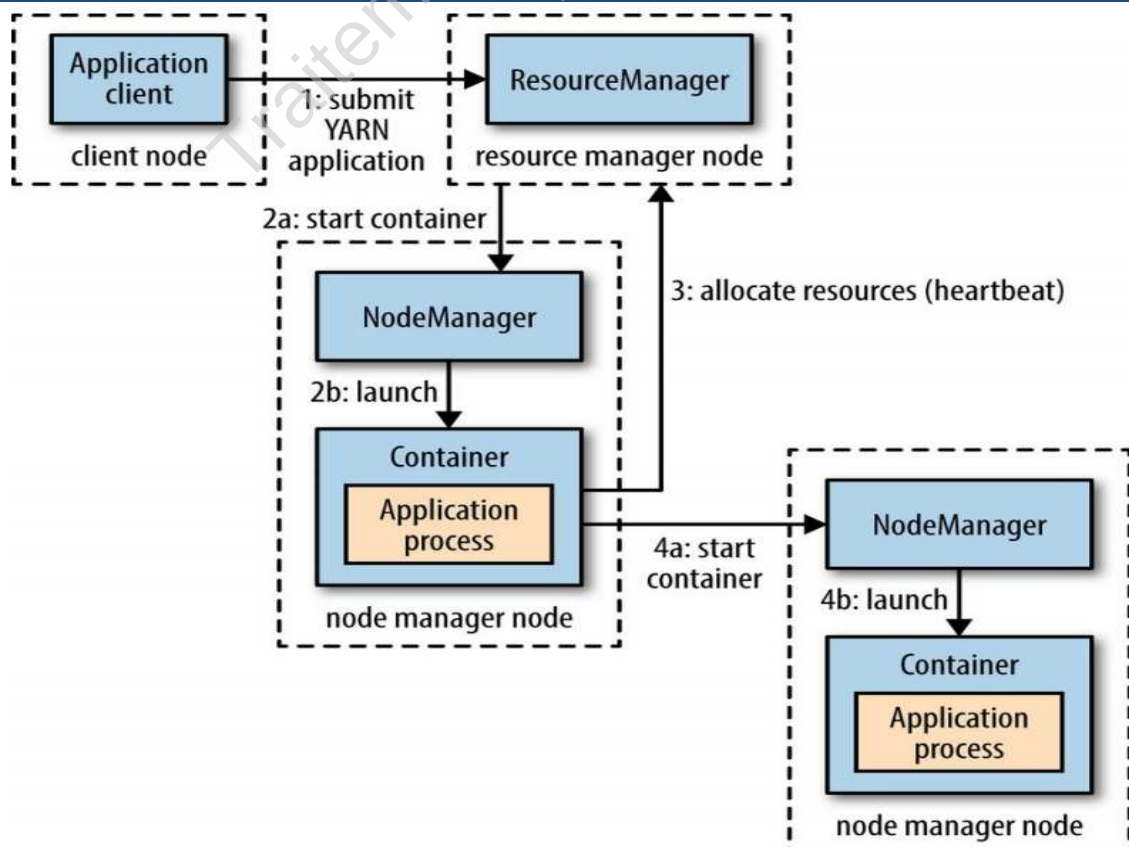
YARN : Architecture et fonctionnement



- MRv2 affecte la charge de gestion des ressources à YARN (Yet Another Resource Negotiator)
- MRv2 sépare la gestion des ressources de celle des tâches MR
- Support les applications MR et non-MR
- Définition de nouveaux demons (Resource Manager, Node Manager, Application Masters)

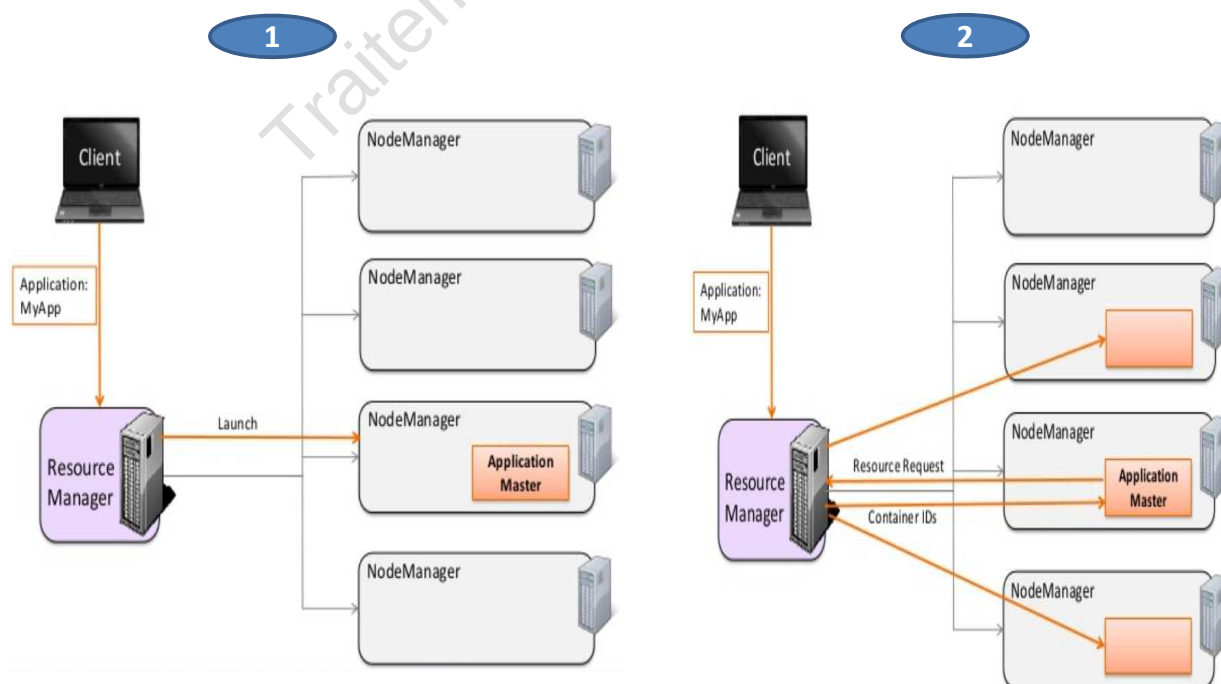


YARN : Architecture et fonctionnement

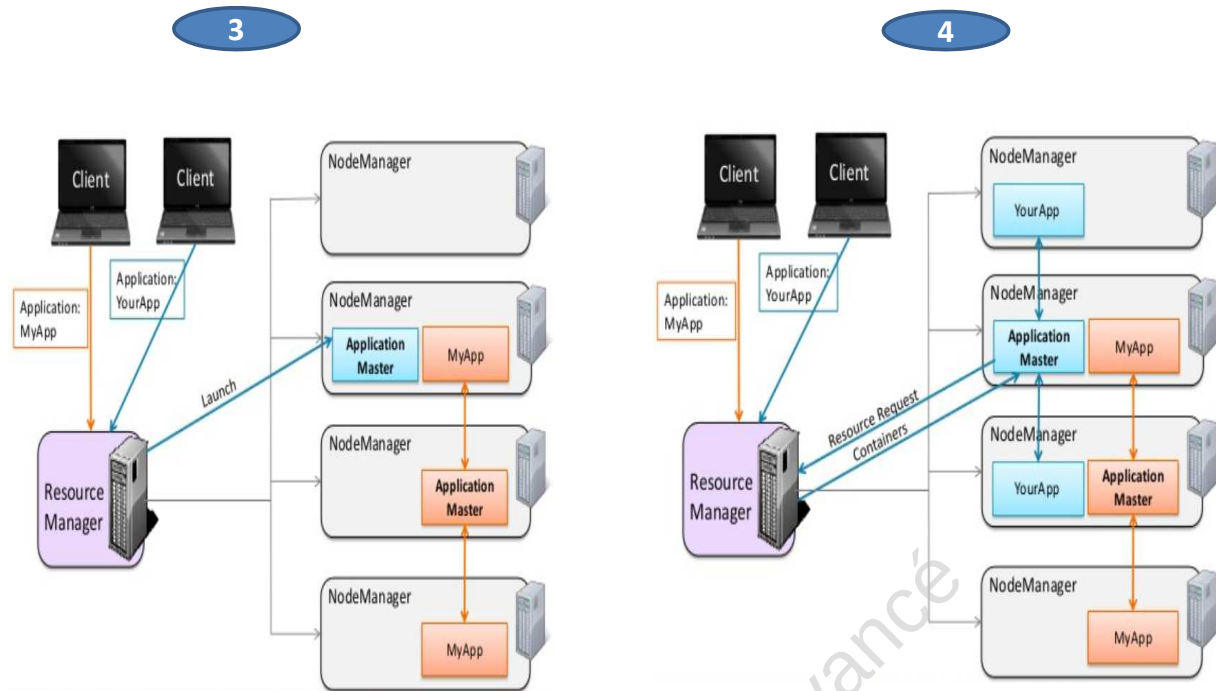


- Pour lancer une application sur YARN, un client contacte le gestionnaire de ressources (**Resource Manager**) et lui demande d'exécuter un processus principal **Application Master** (étape 1).
- **Resource Manager** trouve alors un gestionnaire de nœud (**Node Manager**) qui peut lancer l'application master dans un **conteneur** (étapes 2a et 2b).
- Ce que fait précisément **Application Master** une fois qu'elle est en cours d'exécution dépend de l'application.
 - ✓ Elle pourrait simplement exécuter un calcul dans le conteneur dans lequel elle s'exécute et renvoyer le résultat au client.
 - ✓ Ou elle pourrait demander plus de contenants aux gestionnaires de ressources (étape 3) et l'utiliser pour exécuter un calcul distribué (étapes 4a et 4b)

YARN : MAppReduce Exemple



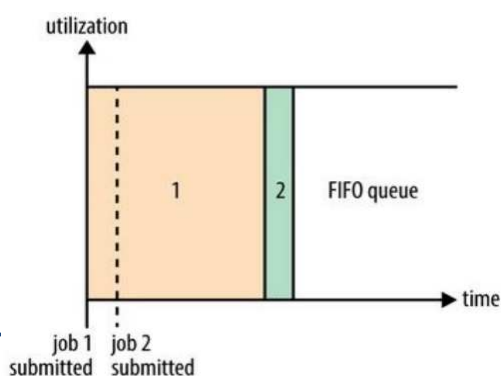
YARN : MAppReduce Exemple



YARN : Allocation des ressources



- Trois planificateurs de ressources sont disponibles dans YARN :
 - le FIFO (First Input First Output)
 - Capacity
 - Fair
- FIFO place les demandes dans une file d'attente et les exécute dans l'ordre de soumission (premier entré, premier sorti).
- FIFO est simple et n'a pas besoin de configuration supplémentaire, mais pas ne convient pas aux clusters partagés. Les grandes applications utiliseront toutes les ressources du cluster, de sorte que chaque application doit attendre son tour.

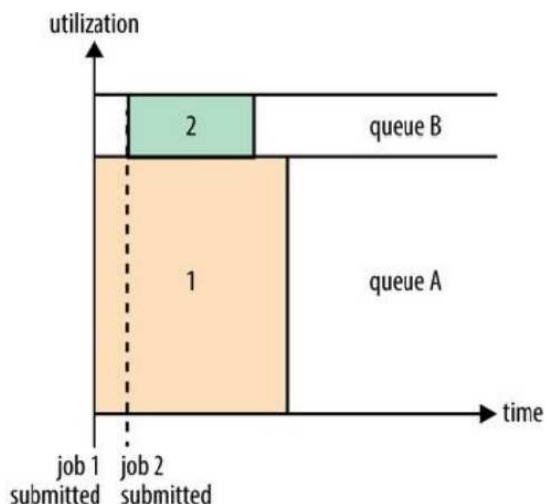


Une petit application (2) doit attendre que l'application (1) gourmande en resource prend fin → latence importante pour l'Utilisateur (2) malgré que son application n'a pas besoin de ressources importantes

YARN : Allocation des ressources

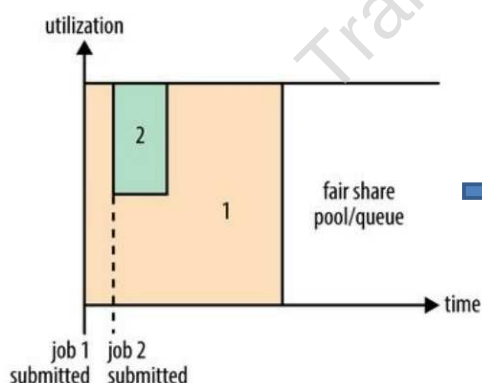


- Dans un cluster partagé, il est mieux utiliser le planificateur (capacity) ou le planificateur (Fair). Ces deux processus permettent aux travaux à long terme de se terminer en temps opportun tout en permettant aux utilisateurs qui exécutent des petites requêtes ponctuelles simultanées d'obtenir des résultats dans un délai raisonnable.



- une file d'attente dédiée distincte permet au petit travail à commencer dès qu'il est soumis. Ceci au détriment de la capacité globale du cluster puisque la capacité de la file d'attente est réservée aux emplois dans cette file d'attente.
- La grande tâche se termine plus tard que lors de l'utilisation du FIFO

YARN : Allocation des ressources



- Avec Fair allocation, il n'est pas nécessaire de réserver une partie fixe de capacité
- Fair équilibrera dynamiquement les ressources entre tous les travaux en cours.
- Juste après le premier (gros) Job commence, (le seul travail en cours d'exécution → il reçoit toutes les ressources du cluster)
- Lorsque le deuxième (petit) travail commence, la moitié des ressources du cluster est allouée au second Job → Exécution immédiate du Job 2
- Lorsque le Job 2 prend fin, toutes les ressources du cluster sont affectées de nouveau au Job 1

Remarque : Allocation Capacity et Fair nécessitent des configurations supplémentaires

■ Facteurs

- Pourquoi exactement nous avons besoin de planifier un cluster Hadoop et quels sont les **différents facteurs** que nous devons examiner, afin de planifier un cluster Hadoop efficace avec une performance optimale ?

- Volume des données

Il est important pour un administrateur Hadoop de connaître le volume de données dont il a besoin pour gérer, planifier, organiser et mettre en place le cluster Hadoop avec le nombre approprié de nœuds pour une gestion efficace des données

- Conservation des données

La conservation des données est un processus où l'utilisateur peut supprimer des données périmées, invalides et inutiles du stockage Hadoop afin d'économiser de l'espace et d'améliorer les vitesses de calcul des cluster.

- Stockage des données

* Le stockage de données est l'un des facteurs cruciaux qui entrent en jeu lors de la planification du cluster Hadoop.

* Les données ne sont jamais stockées directement comme elles sont obtenues. Elles sont soumises à un processus de compression et de chiffrement des données de sorte que la sécurité des données soit assurée et que l'espace utilisé pour enregistrer les données soit le moins important possible.

■ Exigences matérielles pour Hadoop Cluster standard

- L'architecture de Hadoop comprend essentiellement les composants suivants.
 - * Namenode
 - * Job Tracker
 - * Datanode
 - * Task Tracker
- Namenode et Secondaire Namenode sont les parties cruciales de tout Hadoop Cluster. Ils sont attendus à être très disponibles. Les serveurs Namenode et Secondary Namenode sont dédiés au stockage de l'espace de noms et à l'édition-journal.
 - Processor Octa-Core Processor 2.5 GHz
 - RAM 128 GB
 - réseau 10 GBPS
- Presque les même exigences matériels pour DataNode et Task Tracker

■ Exigences relatives au système d'exploitation

- On peut configurer le cluster Hadoop en utilisant un système d'exploitation au choix.
- Les systèmes les plus recommandés pour mettre en place un cluster Hadoop sont
 - * Solaris
 - * Ubuntu
 - * Fedora
 - * Redhat
 - * Centos

▪ Exemple de planification

- Acquisition des données journalières = 100 Go
- Facteur de replication = 3
- Espace non HDFS = 30% (Pour chaque noeud, une certaine capacité de stockage est réservée pour des applications non HDFS)
- Taille de disque dur d'un DataNode = 3 To
- Taille d'un bloc = 128 Mo

- *Nombre de DataNode*

Espace de stockage nécessaire par jour = $100 \text{ Gb} * 3 = 300 \text{ Go}$

Espace de stockage HDFS par noeud = $3 \text{ To} * 0,7 = 2,1 \text{ To}$

Nombre de DataNode (pour une année) = $(365 * 300 \text{ Go}) / 2,1 \text{ To} = 51$

- *Mémoire nécessaire de NameNode*

Mémoire = Espace de stockage (Mo) / taille du bloc (Mo) / 10^6

Mémoire = $365 * 300 \text{ Go} / 128 \text{ Mo} = 876\,000 \text{ octets}$

Remarque : 1 Go / 1 million blocs